

Тема "Города и дороги"

Структура хранения:

Формат 1: Список ребер

5
Moscow
Saint-Petersburg
Novosibirsk
Ekaterinburg
Kazan
0 1 743
0 2 3245
1 3 2356
2 4 1560
3 4 2422

← кол-во городов

инд 0

инд 1

2

3

4

1 и 2 столбик - индекс городов
(откуда-куда)

3 - расстояние

Оба формата
должны подде-
рживаться

Формат 2: Матрица смежности

5
Moscow
Saint-Petersburg
Novosibirsk
Ekaterinburg
Kazan
0 743 3245 0 0
743 0 0 2356 0
3245 0 0 0 1560
0 2356 0 0 2422
0 0 1560 2422 0

5 столбиков

	0	1	2	3	4
	Moscow	S-P	Novosib.	Ekat.	Kazan
0 Moscow	0	743	3245	0	0
1 S-P	743	0	0	2356	0
2 Novosib.	3245	0	0	0	1560
3 Ekat.	0	2356	0	0	2422
4 Kazan	0	0	1560	2422	0

Формат 1 или 2 → загрузка → список смежности

Вика

- создание репозитория
- структура
- main
- Makefile
- презентация
- README
- тесты

Саша

- все ср-и fronted (см. ниже)
- тесты своих ср-й
- документация
- CheckFile

Ваня

- алг. Краскала (+ DSU)
- алг. Прима
- тесты своих ср-й
- сравнение алгоритмов

Ярик

- алг. Дейкстры
- алг. BFS
- загрузка/сохранение графа
- тесты своих ср-й

```
1) // Один эл-т списка стекнотич
typedef struct {
    int number; // номер вершины
    int weight; // вес ребра
    struct EdgeNode *next; // указатель на след. ребро
} EdgeNode;
```

```
2) // Сам граф
typedef struct {
    int numVertices; // всего вершин (городов)
    int numEdges; // всего ребер (дорог)
    EdgeNode **matrix; // Массив указателей на
    // списки стекнотич
    char **cityNames; // названия городов
} Graph;
```


Ярик:

ср-и:

(Также ср-и загрузки должны проверять на корректность формата. Например, что в 1-ой строке всегда есть и т.д.)

- 1) `Graph* CreateGraph(int numVertices)` - создает пустой граф на n вершин
- 2) `void AddEdge(Graph* g, int u, int v, int weight);` // добавляет ребро (ребро) между двумя городами в структуру графа
- 3) `Graph* loadGraphEdges(const char * filename);`
На вход: матрица
// загрузить граф из файла (формат: список ребер)
- 4) `Graph* loadGraphMatrix(const char * filename);`
// загрузить граф из файла (формат: матрица смежности)
- 5) `int SaveGraphEdges(const Graph* g, const char * filename);`
// сохранение графа в файл (формат: список ребер)
- 6) `void FreeGraph(Graph* g);` // очистка памяти графа
- 7) `DijkstraResult* dijkstra(const Graph* g, int start);`
// Алгоритм Дейкстры. Вход: структура с 2-мя массивами: `dist` и `prev`
- 8) // Структура результата
`typedef struct {`
 `int *dist; // массив расстояний`
 `int *prev; // массив предыдущих вершин (для восстановл. пути)`
 `int source; // исходная вершина`
`} DijkstraResult;`
- 9) `int* GetPath(const DijkstraResult* res, int target, int *pathlen);`
// Восстановл. пути из результата Дейкстры, возвращает массив с вершинами (Москва → Питер → Новосибир.)
- 10) `void FreeDijkstraResult(Dijkstra* res);` // очистка памяти
- 11) `int* BFS(const Graph* g, int start, int *orderlen);`
// Алгоритм BFS. Возвращает массив вершин в порядке обхода
- 12) `void PrintGraph(const Graph* g)` // для отладки

Ваня: 1) `MSTResult* kruskal (const Graph* g)` // прои

мимит. отсловное дерево. Вывод: структура с ребрами

MST, общим весом и кон-вом ребер

2) `MSTResult* prim (const Graph* g)` // тоже самое, только алг. Прима

3) `void FreeMSTResult (MSTResult* res)` // освободе. памяти

4) `DSU* DSUCreate (int n)` // создать сист. непересекающихся мн-в

на n эл-ов. Выход: указатели на структуру DSU

5) `void DSUFree (DSU* dsu)` // освободе. памяти

6) `int DSUFind (DSU* dsu, int x)` // находит корень мн-ва, содержащ.

вершину x. Выход: корень мн-ва

7) `void DSUUnion (DSU* dsu, int x, int y)` // объединяет два

мн-ва, содержащие x и y

```
typedef struct {  
    int totalWeight;
```

```
    int *u, *v;
```

```
    int *weights;
```

```
    int numEdges;
```

```
} MSTResult;
```

8) `double GetTime ();` // замер времени

9) `Comparison CompareMST (const Graph* g);` сравнение алгоритмов MST

```
typedef struct {
```

```
    double timeKruskal;
```

```
    double timePrim;
```

```
    int totalWeight;
```

```
    int numEdges;
```

```
} Comparison;
```

10) `FreeGraph`

Саша: 1) int Menu()

Должно вводить что-то типа:
// GraphMaster: Планировщик маршрутов
и анализатор сетей
Главное меню:
1. Загрузить граф из файла
2. Найти кратчайший путь (Дейкстра)
3. Построить MST (Краскал)
4. Построить MST (Прим)
5. Обход графа в ширину (BFS)
6. Сравнить алгоритмы MST

Выберите действие:

2) char * AskFilename() // вводит, введете путь к файлу с
графом; " и возвращает указатель на строку

3) void PrintDijkstra(^{Graph* g} const int *path, int pathlen, int totalWeight)
// вводит: Результат Дейкстры
Кратчайший путь: (пути он вообще не существует)
Длина всего пути: (должно вводить по типу: Москва → Новосибирск?)

4) void PrintMST(const MSTResult *res)
// вводит: Минимальное остовное дерево (MST)
Ребра MST: Москва - Новосибирск 3200 км
.....
Общий вес дерева:

5) void PrintBFS(Graph* g, int *order, int orderlen)
// вводит: Обход в ширину (BFS)
Порядок обхода вершин: (Москва → Новосибирск → ...)
Посещено городов: (много)

- еще можно сделать
таба рядом с городами
или отделить баки
подписями индекса

6) void PrintGraphInfo(int numVertices, int numEdges)
// Вершин: Информация о графе после загрузки
Ребер:

7) `int CheckFile(const char *filename)` // проверка на существование

8) `void PrintComparison(const Comparison* cmp);` - вывод сравнения
// тоже в виде табл.;

<u>Сравнение алгоритмов</u>		
Алгоритм	Время(мс)	Вес